

SOFTWARE DESIGN DOCUMENT

AI Meeting Intelligence System

Digital Assignment 2 / Review 2

Frontend	React + Vite
Backend	Node.js + Express.js
AI	AI summarization / extraction service
Data	Database persistence layer
Development	VS Code • Git • GitHub • Docker

Replace the marked diagram and Figma placeholders with your final exports before submission.

1. Design Principles Applied

Abstraction

The UI communicates through simple component props and REST APIs. Database and AI implementation details remain behind service boundaries.

Modularity

Frontend screens, reusable components, API services, backend routes and data access are separated into focused modules.

High Cohesion

Related responsibilities stay together: authentication handles authentication, meeting modules handle meetings, and task modules handle tasks.

Low Coupling

Modules communicate through stable interfaces rather than depending on each other's internal implementation.

Separation of Concerns

Presentation, business logic, persistence and external AI processing are treated as separate concerns, improving testing and maintainability.

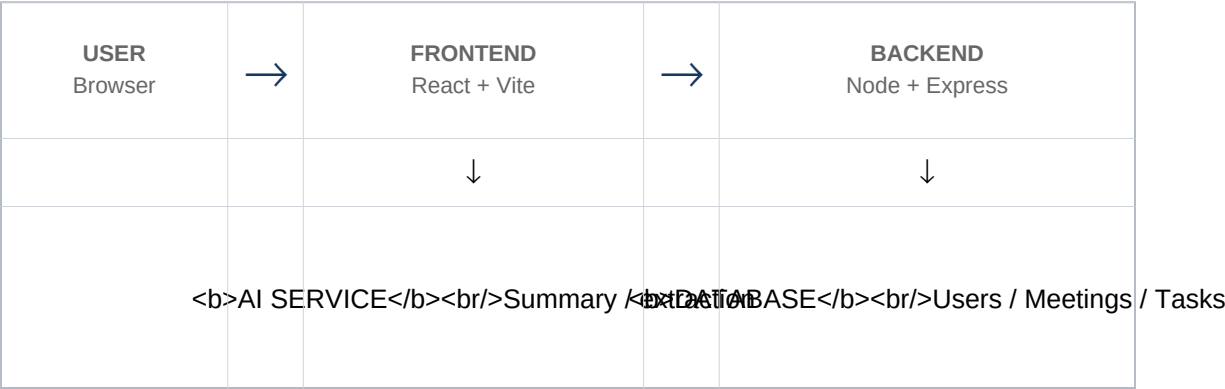
Example structure

```
src/  
  components/  reusable UI  
  pages/       screen-level views  
  services/    API communication  
  hooks/       reusable logic  
  assets/      static assets
```

2. High-Level Architecture

Architecture style: layered client-server architecture.

This style was chosen because the system naturally separates the presentation layer, REST API/business logic, persistence, and AI processing. Clear boundaries reduce change impact and make the system easier to maintain.



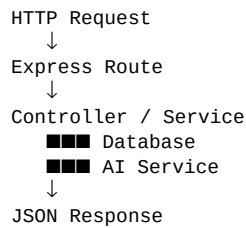
Containerization: Frontend and backend can run as Docker services. **Deployment:** the same services can be deployed to a cloud/VM environment with environment-specific configuration.

FINAL DRAW.IO PNG EXPORT



3. Backend Design & Code Snippets

The backend provides the central API boundary between the React interface, data layer, and AI service. This prevents the UI from depending directly on internal persistence or AI implementation.



Example route

```
router.get("/meetings", async (req, res) => {
  try {
    const meetings = await meetingService.getAll(req.user.id);
    res.json(meetings);
  } catch (error) {
    res.status(500).json({
      message: "Unable to load meetings"
    });
  }
});
```

The route handles HTTP behavior while the service owns meeting logic. This improves cohesion and allows the implementation to change without rewriting the route or UI.

Example AI service boundary

```
async function generateSummary(transcript) {
  const result = await aiClient.createSummary(transcript);
  return result.summary;
}
```

4. Frontend Design & Code Snippets

The frontend uses reusable React components and screen-level pages. Repeated patterns such as navigation, cards, task rows and status indicators can be reused across screens.

Reusable component

```
function MeetingCard({ meeting, onOpen }) {
  return (
    <article className="meeting-card">
      <h3>{meeting.title}</h3>
      <p>{meeting.date}</p>
      <button onClick={() => onOpen(meeting.id)}>
        View details
      </button>
    </article>
  );
}
```

API service

```
import axios from "axios";

export const getMeetings = async () => {
  const { data } = await axios.get("/api/meetings");
  return data;
};
```

Why this helps

Reusable components reduce duplication and visual inconsistency. A centralized API service also keeps network communication separate from presentation code, making future API changes easier.

5. User Interface Design

The six core screens follow the main user journey: authenticate → view meetings → upload transcript → inspect meeting intelligence → manage tasks → manage profile.

Login

INSERT FINAL FIGMA EXPORT — LOGIN

Dashboard

INSERT FINAL FIGMA EXPORT — DASHBOARD

Upload Transcript

INSERT FINAL FIGMA EXPORT — UPLOAD TRANSCRIPT

Meeting Details

INSERT FINAL FIGMA EXPORT — MEETING DETAILS

Tasks

INSERT FINAL FIGMA EXPORT — TASKS

Profile

INSERT FINAL FIGMA EXPORT — PROFILE

6. Design Decisions & Maintainability

REST API boundary

Provides a stable contract between frontend and backend and allows additional clients to be added later.

Reusable React components

Reduces duplicated UI code and makes global interface changes faster.

Service boundaries

Keeps database and AI-provider details isolated, reducing coupling.

Docker-based development

Creates a repeatable environment and reduces machine-specific setup differences.

Centralized error handling

Provides understandable feedback to users while keeping technical diagnostics available for debugging.

Maintainability & future changes

The design uses small modules with clear responsibilities. A developer can change a screen, API implementation, AI provider, or persistence layer without redesigning the whole application. Future additions such as richer analytics, notifications, new export formats, role-based access, or alternative AI providers can therefore be introduced incrementally.

7. Repository & Submission Checklist

GitHub Repository: INSERT YOUR ACTUAL REPOSITORY LINK

Recommended design folder

```
design/
  ■■■ architecture/
  ■ ■■■ AI-Meeting-Architecture.drawio
  ■ ■■■ AI-Meeting-Architecture.png
  ■■■ figma/
  ■ ■■■ login.png
  ■ ■■■ dashboard.png
  ■ ■■■ upload-transcript.png
  ■ ■■■ meeting-details.png
  ■ ■■■ tasks.png
  ■ ■■■ profile.png
  ■■■ Software-Design-Document.pdf
```

README — Software Design section

```
## Software Design
```

```
The AI Meeting Intelligence System follows a modular client-server
architecture with a React/Vite frontend and Node.js/Express backend.
The design separates presentation, business logic, API communication,
and data persistence to improve maintainability and reduce coupling.
```

```
### Architecture
```

```
[View Architecture Diagram](./design/architecture/AI-Meeting-Architecture.png)
```

```
### UI Design
```

```
Six primary screens cover Login, Dashboard, Upload Transcript,
Meeting Details, Tasks, and Profile.
```

Submission checklist

- Final PDF is 6–10 pages.
- Editable Draw.io source and PNG export are in /design/.
- Six final Figma exports and prototype link are documented.
- README contains the Software Design section.
- Actual GitHub repository link is included in the report.